ORBIT PLATFORM · TECHNICAL REFERENCE

Enterprise Web Platform

The corporate travel console. React 19, TypeScript, Tailwind 4 and TanStack, with design tokens generated from Figma and every call routed through the gateway.

Repository	orbit-enterprise-web
Kind	Client
Ports	5173
Tests	23 passing
Platform	React 19

Generated from the service source. Regenerate rather than editing.

Contents

1 Overview

- Responsibilities
- Non-responsibilities

2 Everything goes through the gateway

3 Design tokens come from Figma

- Colours are named by role
- Focus is always visible

4 Money

5 The access token lives in memory

- Expired thirty seconds early
- Concurrent refreshes collapse into one

6 Error handling

7 Retry policy

- Mutations are never retried automatically

8 Query keys in one place

9 Booking

- The response must be branched on

10 Approvals

11 The trip log

12 Architecture

- DataList
- The shell is persistent

13 Theme

14 TypeScript configuration

- no-floating-promises is an error

15 Container

- pnpm blocks postinstall scripts

16 Running it

17 Testing

Overview

`orbit-enterprise-web` is the corporate travel console: booking against a company account, ride policy, cost centres, approvals and consolidated billing.

React 19, TypeScript, Tailwind 4, TanStack Query/Router/Table, Vite, pnpm.

Responsibilities

- Booking within — or outside, with approval — the employee's ride policy
- The manager's approval queue
- Employee, cost centre, policy and invoice administration
- The trip log

Non-responsibilities

- **Any business rule.** Policy is evaluated by the enterprise BFF; the client renders the verdict
- **Holding a service URL.** Every call goes through the gateway
- **Deciding what a fare is**

Everything goes through the gateway

There is **no service URL anywhere in this codebase**, in any environment.

```
browser ──► /api  ──► Vite proxy (dev) / nginx proxy (container) ──► YARP gateway
```

> **WHY** The gateway is where authentication, rate limiting and header sanitisation happen. A client that can reach a service directly is a client that can skip all three — and once one environment's config names a service host, that config gets copied.

Same-origin `/api` also means **no CORS** in production. The browser only ever sees its own origin, which removes a whole class of preflight and credential-mode problems.

Design tokens come from Figma

`src/design/tokens.generated.css` is extracted from the Orbit design system file — 51 colour tokens per theme, plus the spacing, radius, size and type scales.

> **WARNING** It must not be edited by hand. Regenerate it. A hand-tweaked value diverges silently from what the designer sees, and nothing anywhere reports the disagreement.

`theme.css` maps those tokens onto Tailwind 4's `@theme`, so `bg-surface` and `--bg-surface` are the same thing rather than two things that happen to agree today.

There is deliberately **no `tailwind.config.js`**. Tailwind 4 configures itself in CSS, and having the theme in two places is how a token and a utility class drift apart.

Colours are named by role

`--color-brand`, not `--color-blue-600`.

> **DECISION** `bg-brand` survives a rebrand; `bg-blue-600` becomes a find-and-replace across two applications. The same reasoning gives `--color-status-arrears` a name rather than a hue — "arrears" is a concept the platform has, and the colour is an implementation of it.

Focus is always visible

```
:where(a, button, input, select, textarea, [tabindex]):focus-visible {
  outline: 2px solid var(--border-focus);
  outline-offset: 2px;
}
```

Never removed. Operations and finance staff work these consoles at speed and largely by keyboard.

Money

Minor units all the way from the ledger to the `<Money>` component, which divides at the last moment before a human reads it.

```
new Intl.NumberFormat('en-NG', { style: 'currency', currency, minimumFractionDigits: 2 })
  .format(minorUnits / 100)
```

> **WHY** Every earlier conversion to a float is a rounding error waiting to be reconciled, and a ledger that disagrees with a receipt by one kobo is a ticket nobody can close.

Amounts render in **tabular figures** and right-align in tables. A fare column with proportional digits cannot be scanned, and scanning is the whole job on a finance screen.

Zero renders as `0.00`, not as blank. A fully refunded ride is a real amount and must not look like a missing value. A null budget renders as an em dash, because "no budget set" and "a budget of nothing" are different facts.

The access token lives in memory

A module-scoped variable. **Never** `localStorage`.

> **WARNING** Anything in local storage is readable by any script that ends up on the page, which includes whatever a supply-chain compromise in a dependency puts there. Losing the token on reload is the cost; the httpOnly refresh cookie pays it.

Expired thirty seconds early

```
expiresAt = Date.now() + (expiresInSeconds − 30) ∗ 1000
```

So a token does not lapse in flight between the check and the server reading it. Clocks drift and networks are slow.

Concurrent refreshes collapse into one

```
refreshInFlight ??= performRefresh().finally(() => { refreshInFlight = null })
```

> **WARNING** Ten queries failing with 401 at the same moment is the normal case after expiry. Ten parallel refreshes rotate the token family ten times, which the identity service correctly reads as **reuse** — and answers by revoking the whole family. The user is signed out for no reason, and the cause looks like an identity bug rather than a client one.

A refresh is attempted **exactly once** per request. A refresh that itself fails means the session is genuinely over, and retrying in a loop turns an expired login into a burst of traffic against identity.

Error handling

`ApiError` carries the server's own account of the failure.

```
class ApiError extends Error {
  readonly status: number
  readonly code: string          // "ride.already_cancelled"
  readonly traceId?: string
  readonly fieldErrors?: Record<string, string[]>
  get isRetryable(): boolean { return this.status === 429 || this.status >= 500 }
}
```

The UI branches on **`code`**, never on `detail`. The detail is prose written for a person and may be reworded at any time; a client that branches on it breaks on a copy change.

A body that is not a problem document — a gateway's HTML error page, an empty response — falls back to the status line. Inventing a code there would send the UI down a branch built for a different failure.

Retry policy

```
retry: (failureCount, error) =>
  error instanceof ApiError ? error.isRetryable && failureCount < 2 : failureCount < 2
```

DECISION TanStack's default retries everything three times. That turns one 403 into four and makes an authorisation bug look like a slow page. It also makes a 429 worse — the rate limit is the reason for the failure, and hitting it three more times extends the penalty.

A **409 is not retryable**: the state moved on, and repeating the same request against the same stale assumption produces the same conflict forever.

Mutations are never retried automatically

WARNING The mutations that matter move money. A silent retry of a request whose response was lost is a second charge. Retrying is the user's decision, with the same idempotency key — which is what makes it safe.

Query keys in one place

```ts
export const queryKeys = {
  rides: { detail: (id: string) => ['rides', 'detail', id] as const, ... },
  approvals: { queue: () => ['approvals', 'queue'] as const },
}
```

WHY Invalidation is only correct if the key that reads and the key that invalidates are built the same way. Two hand-written arrays that differ by one element produce a screen that never refreshes, and nothing anywhere reports an error.

Booking

The quote and the policy verdict arrive **together**, per vehicle class.

A ride the policy forbids is **not selectable at all**, with the reason shown next to it.

> DECISION Telling an employee their ride is confirmed and then that it needs their manager's approval is how a travel tool loses the people who use it daily — and the reliable consequence is that they book personally and expense it, which costs the company both the corporate rate and the visibility the policy existed to provide.

Surge is disclosed **on the option, before booking**. A fare higher than quoted with no warning is the single most complained-about thing in ride hailing.

The idempotency key is generated **when the user commits**, not on render. A key regenerated on each re-render makes a retry a second ride.

The response must be branched on

```
book.data.status === 'AwaitingApproval'
  ? 'Sent to your manager. You will be notified when it is decided.'
  : 'Booked. Your driver is being found now.'
```

Telling an employee their ride is on the way when it is sitting in a queue is the worst outcome this flow produces.

Approvals

The queue **refetches every 30 seconds** on its own.

> WHY An employee is standing on a pavement waiting for this decision. A queue that only updates when the manager reloads is one where the answer arrives after it stopped mattering.

`onSettled` invalidates whether the decision succeeded or failed. On failure the row may have been decided by somebody else, and leaving it on screen invites a second attempt at something already done.

A 409 renders as "Someone else has already decided that request" rather than a generic error, because that is a normal outcome of two managers opening the same queue.

The trip log

Cursor-paged, not offset.

> **WHY** A trip log is written to continuously. With `OFFSET`, a row inserted between page one and page two shifts everything down by one, and the user sees a row twice while another disappears entirely.

`placeholderData: keepPreviousData` keeps the previous page on screen while the next loads. Without it the table collapses to empty on every page turn and the layout jumps.

An unrecognised ride state maps to a neutral pill rather than throwing.

> **WHY** A newer backend can add a state. An older console must keep rendering the table rather than crashing on an index that is not there.

Architecture

```
src/
  app/          router, AppShell
  design/       tokens.generated.css (DO NOT EDIT), theme.css
  lib/
    api/        client, problem details
    auth/       session
    query/      query client, query keys
  components/ui/  Button, Money, StatusPill, ThemeToggle, cn
  features/
    auth · dashboard · booking · approvals
    employees · policy · finance · trips · shared/DataList
```

`DataList`

Six pages of hand-written `<table>` markup drift: one forgets a `scope`, another loses the empty state, a third right-aligns money and a fourth does not. One implementation means fixing accessibility once.

A column is either `render` or `money`, never both — amounts get right alignment and tabular figures **automatically**, because a money column somebody forgot to align is a column nobody can scan, and forgetting is the default when it is opt-in.

The shell is persistent

A persistent sidebar rather than a router-driven layout per page, so navigating does not unmount and remount the navigation. Rebuilding the chrome on every route change is slower and visibly flickery on a slow connection.

`min-w-0` on the content column and its flex parent. Without it a wide table forces the whole layout wider instead of scrolling inside its own container, and the sidebar slides off screen.

Theme

Applied **before first paint** by an inline, synchronous script in `index.html`.

> **WHY INLINE** A deferred script runs after the browser has already painted, so a dark-mode user sees a white flash on every load.

The toggle reads its initial value from the **DOM**, not from storage, because the bootstrap has already resolved it — including the system preference for a user who has never chosen. Reading storage again would disagree with what is on screen for exactly those users.

TypeScript configuration

Every strictness flag is on.

FLAG	CATCHES
`strict`	The baseline
`noUncheckedIndexedAccess`	`array[0]` being `undefined`
`exactOptionalPropertyTypes`	`undefined` assigned where a property is merely optional
`noImplicitOverride` · `noFallthroughCasesInSwitch`	
`noUnusedLocals` · `noUnusedParameters`	Dead code
`verbatimModuleSyntax` · `isolatedModules`	Bundler correctness

`exactOptionalPropertyTypes` is the one that bites: `fetch`'s `body` is `BodyInit | null`, not `BodyInit | undefined`, so the API client passes `raw ?? null`. That is a real distinction — `null` is what fetch means by "no body".

`no-floating-promises` is an error

> **WARNING** A floating promise in a mutation handler is a request that silently never completed, and the user is left looking at a spinner. It is the single most valuable lint rule in this codebase.

Type-aware rules are scoped to `**/*.{ts,tsx}`. Spreading them at the top level makes ESLint try to type-check its own config file, which is not in the tsconfig and never will be.

Container

Multistage. The build toolchain — Node, pnpm, TypeScript, ~400 MB of `node_modules` — exists only in the first stage.

> **WARNING** A production image containing a compiler is a production image containing an attacker's toolchain.

What ships is static files and nginx running as **UID 64198**.

PATH	CACHE
`/assets/*`	`1y, immutable` — filenames are content-hashed
`/index.html`	`no-store`

> **Why index.html is never cached:** It is the file that names the current assets. A stale copy pins a user to a deployment that no longer exists, and the symptom is a blank page after a release.

The gateway URL is substituted at **startup** through the nginx image's `envsubst` template mechanism, filtered to `ORBIT_GATEWAY_URL` only.

> **WARNING** Without the filter, `envsubst` also eats `$host`, `$scheme` and `$uri` — nginx's own runtime variables — and the proxy then forwards an empty `Host` header to every upstream. A plain `.conf` file would not be substituted at all, and the proxy would target the literal string `${ORBIT_GATEWAY_URL}`.

pnpm blocks postinstall scripts

`pnpm-workspace.yaml` allows exactly two: `esbuild` places a platform binary, `msw` writes its service worker.

> **WHY** An arbitrary script running at install time is the easiest way for a compromised transitive dependency to reach a developer's machine. Everything else stays blocked.

Running it

```
cp .env.example .env.local
pnpm install
pnpm dev
```

http://localhost:5173. The gateway must be running.

COMMAND	
`pnpm dev`	Vite, proxying `/api`
`pnpm build`	**Typecheck, then bundle.** A build that skips the typecheck is one where the types stopped meaning anything
`pnpm test`	Vitest
`pnpm lint`	Type-aware ESLint

Testing

23 tests.

`ApiError` — that it branches on the stable code; that a 409 is not retryable while 429 and 5xx are; that a 403 is not retried; that a non-problem body falls back to the status line; that field errors surface.

Session — that the token is never in storage; that it expires thirty seconds early; that **concurrent refreshes collapse into one request**; that a rejected or failed refresh clears the session.

Components — that a loading button cannot be clicked twice; that it reports `aria-busy` rather than `disabled` semantics; that `twMerge` resolves a conflicting class; that `Money` divides only at render and uses tabular figures; that a status pill never relies on colour alone.